

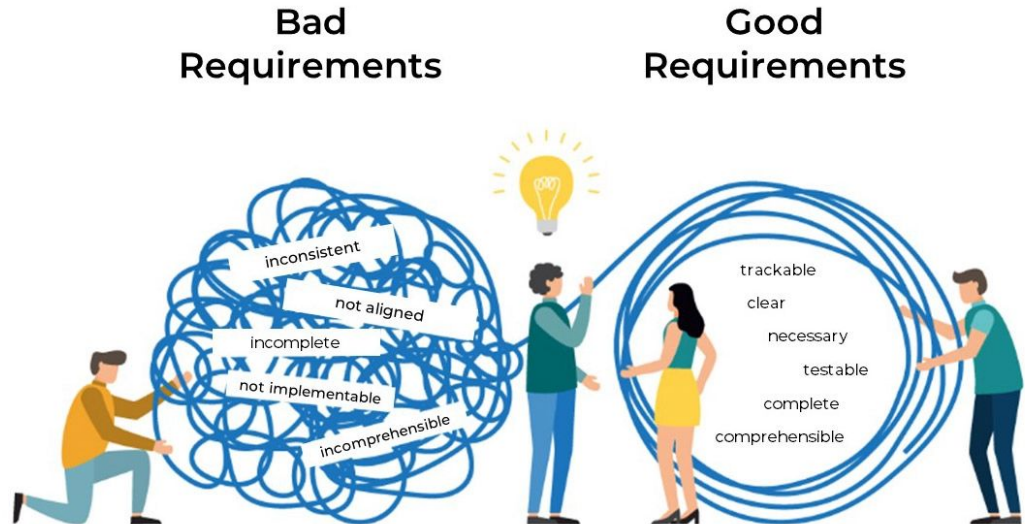
COMS W4156 Advanced Software Engineering (ASE)

October 24, 2023



Agenda

1. Software Requirements Overview
2. User Stories
3. Use Cases

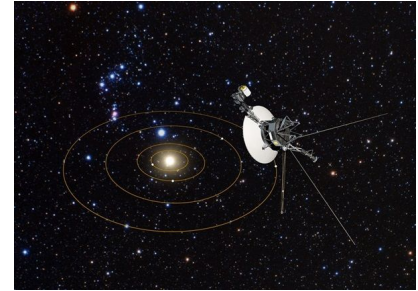


Software Requirements

Some requirements are specified in immense detail, e.g., see [HTTP](#), [URL](#) RFCs

Such "specifications" are necessary for interoperability across large numbers of different vendors, e.g., [so internet software can talk to each other](#)

Specifications are also necessary for safety-critical software and any other circumstances where you must get it right the first time, e.g., [NASA Voyager Telecommunications](#)



In this course we do not consider the detailed specifications necessary for safety-critical software or interoperability, only "informal" requirements

Customers vs. Users

Informal approaches to defining requirements are usually phrased in terms of what "customers" or "users" want, where "users" typically means end-users

Customer is not necessarily the same as user

If you are developing software to control a self-driving car, your customer might be Waymo or Tesla, but your primary user is the person sitting behind the wheel of the car - that user is your customer's customer

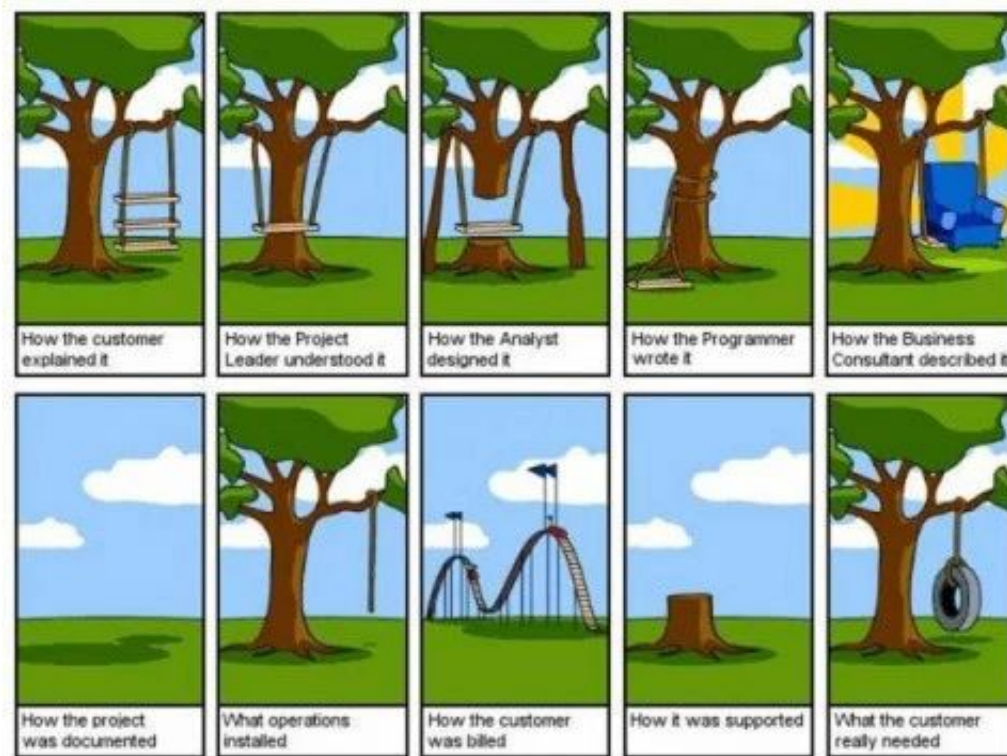
What If There Aren't Any Customers/Users (Yet)

Many software teams have internal “product owners” who proxy for customers/users

Even in a startup or other new product venture with no customers/users yet, someone has an *idea* about what will attract customers by providing *value* - and (usually) has a business model as to how this will enable paying for the development and deployment effort

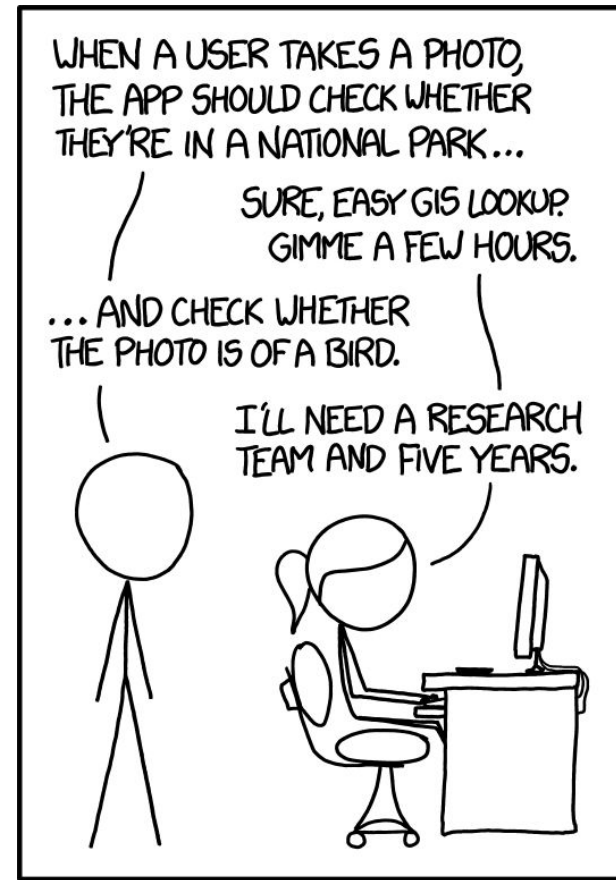
The terms customer and user are often used interchangeably, not distinguishing whether requirements come from customers/users, from their customers/users, or from a product owner

Customers Don't Always Know What They Want Or How To Describe It



Customer Requirements Are Not Always Realistic

[This cartoon is out of date](#)



IN CS, IT CAN BE HARD TO EXPLAIN
THE DIFFERENCE BETWEEN THE EASY
AND THE VIRTUALLY IMPOSSIBLE.

Requirements Elicitation

- Brainstorming workshops
- Observation and interviews - constraints on how the software should fit into business workflows and how it interacts with pre-existing software or manual procedures
- Role playing - pretend to be the software interacting with the user

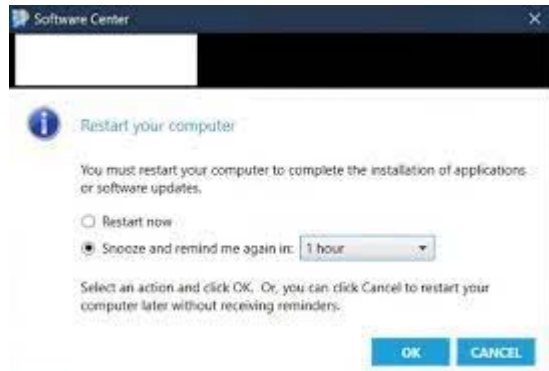


Most Software Requirements Change

Almost all modern business and consumer software is frequently updated

Google and Facebook update their user-facing software multiple times **per day**

These organizations are not updating RFC-like documents every day, they use shorter, simpler requirements documents and processes

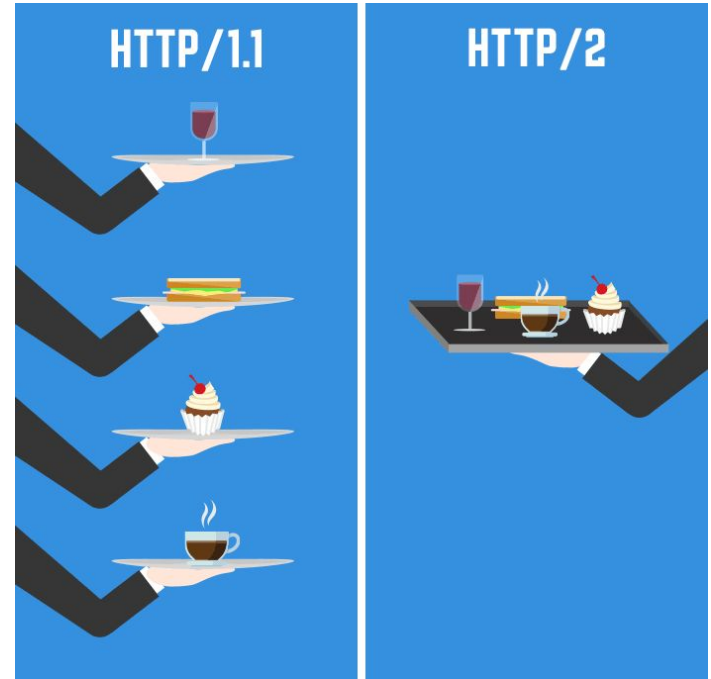


Even RFCs Change

Even detail specifications change:

HTTP/0.9 $\Rightarrow$ HTTP 1.0 $\Rightarrow$ HTTP/1.1
 $\Rightarrow$ HTTP/2 $\Rightarrow$ HTTP/3

Slow evolution of detailed internet standards is enabled by the [IETF standards process](#), but it (intentionally) cannot support rapid change



Agenda

1. Software Requirements Overview
2. User Stories
3. Use Cases

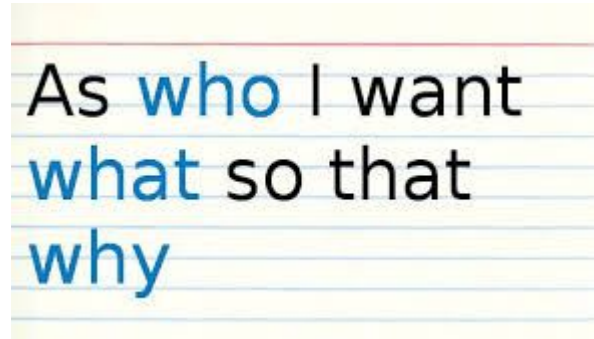
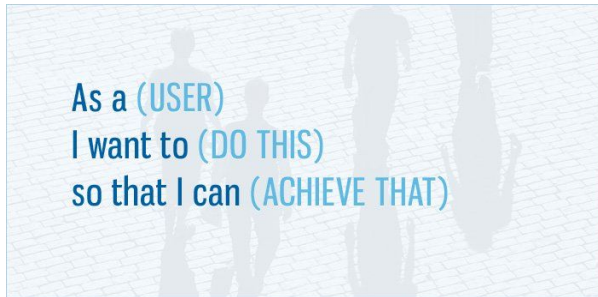


User Story ~= Feature

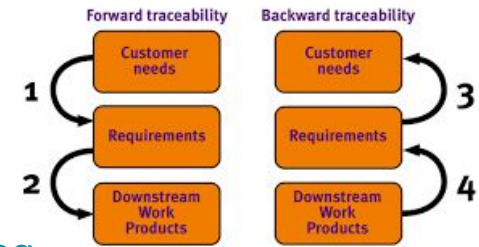
User stories define informal requirements in terms of what users will be able to do with the software that they couldn't do without it, or couldn't do as well without it, that provides them some *value*

A user story is a short simple description of ONE feature, ideally fitting on a 3x5 card, written in the user's language (domain terminology)

< label >: As a < type of user >, I want < some goal > so that < some reason >



User Story Elements



Why label? Title useful for cross-referencing, e.g., [requirements tracing](#)

Why type of user? Software often has multiple *roles* that use the same program differently, such as performing different tasks (e.g., buyer vs. seller, doctor vs. patient, instructor vs. TA vs. student)

- ❖ In the case of client apps for a service, different roles do not imply different clients - the same client often has many different user roles. For example, Courseworks has [13 roles](#) and pulls data from the registrar API, but it's one client (with up to ~40k instances) not 13 different clients. However, Vergil is a different client wrt the registrar API

Why goal? What the user wants, what the feature is supposed to do for the user

Why reason? Why the user wants it, how the feature will provide value

Example User Stories

As a PC user, I want to backup my entire hard drive so that my files are safe

? Is this enough information to start developing the backup program ?

Every non-trivial application has multiple user stories, not just one!



As a PC user, I want to specify which folders to backup to make sure my most important files are saved but my backup store isn't filled with things I don't need

The same role may have multiple requirements, even mutually incompatible requirements - so the software needs to provide a way for the user to choose which one to do



More Example User Stories

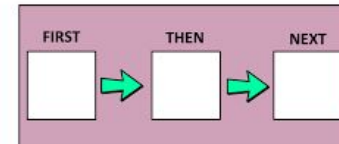


As a PC user, I want to backup my entire hard drive so that my files are safe

As a PC user, I want to specify which folders to backup to make sure my most important files are saved

As a PC user, I want to specify subfolders and filetypes to skip within the backup folders so my backup store isn't filled with things I don't need

The same role may have multiple requirements, some of which may be mutually incompatible while some depend on others - so the software needs to provide a way for the user to first choose to do this, then choose to do that



Turning User Stories into Software

What other information might the developer need to start coding? - Another way to think of it is what additional information will the backup program need?

What permissions will the backup program need to access the files its supposed to backup

Where to put the backup copies of the files

What to do if the backup store is not available

What to do if the backup store is full

What to do with the files already in the backup store

Etc...



SMART Conditions of Satisfaction

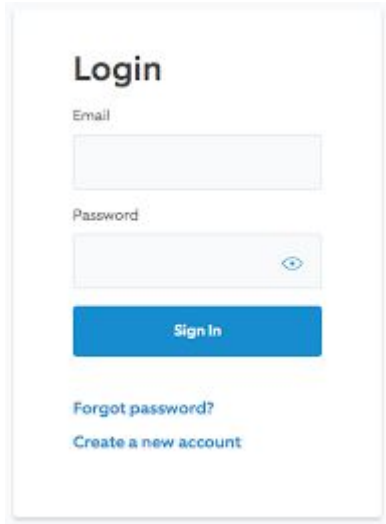


Conditions of satisfaction ideally fit on the back of the 3x5 card

< label >: As a < type of user >, I want < some goal > so that < some reason >

My conditions of satisfaction are < list of common cases and special cases that must work >

Do These Conditions of Satisfaction Say Enough to Start Coding?



The image shows a login form with the following elements:

- Login** (Section Header)
- Email** (Label) with a text input field.
- Password** (Label) with a text input field and a toggle icon (eye) to show/hide the password.
- Sign In** (Button) in a blue box.
- [Forgot password?](#) (Link)
- [Create a new account](#) (Link)

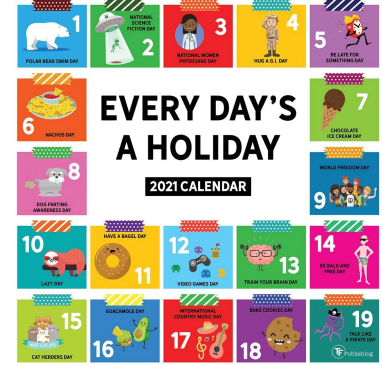
As a user, I am required to login before using the site

Conditions of Satisfaction:

- user is logged in only when proper credentials are provided
- a "remember me" option is available
- user can request a password reminder
- user is locked out after three failed attempts

What About These?

As a vice president of marketing, I want to select a holiday season to be used when reviewing the performance of past advertising campaigns so that I can identify the most profitable ones



Conditions of Satisfaction:

- Make sure it works with major retail holidays: Christmas, Easter, President's Day, Mother's Day, Father's Day, Labor Day, New Year's Day.
- Support holidays that span two calendar years (Christmas through New Year's).
- Holiday seasons can be set from one holiday to the next (Thanksgiving to Christmas).
- Holiday seasons can be set to be a number of days prior to the holiday.

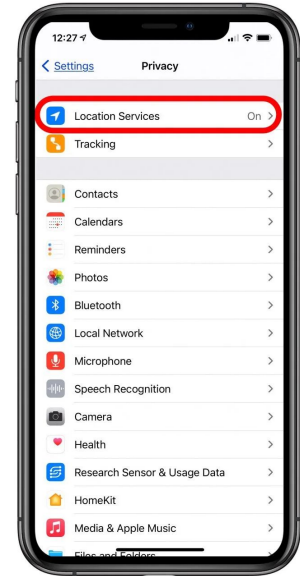
Does This Story Say Enough to Start Coding?

As a traveler, I want to know which nearby shops participate in my rewards program, so I can buy stuff

Conditions of Satisfaction:

- If my location service is not already turned on, it should prompt me to turn it on
- It should allow me to input a location address instead

? What else does developer need to know ?



Expanding User Roles to Personas

Synthetic biographies of fictitious users, usually one for each category likely to have different preferences and expectations of the product - possibly more than one persona per role

“What would Jeremy do with this app?”

“Will Scott & Melissa understand this interaction or how to use this feature?”

Personas help find and fill gaps in collection of known user stories



The Weekender

Jeremy, 30

This trip will be the first excursion he's taken with his credit card rewards. Jeremy is rewarding himself for being financially responsible. He's constantly on Instagram and Facebook and brags to his friends about booking the trip with his reward points.

Interests

- Adventure
- Luxury
- Wellness
- Nightlife
- Food

Travel Habits

- Weekends
- Flying & road trips
- Stays with friends
- Uber



The Points Gurus

Scott & Melissa, 60 & 58

With a well-established line of credit and a long credit history, Scott and Melissa have accumulated points from rewards programs over the years. As a recently retired couple, they are looking to travel more. While they feel comfortable using smartphone, but feel behind the times.

Interests

- Luxury
- Wellness
- Food
- Outdoors
- History

Travel Habits

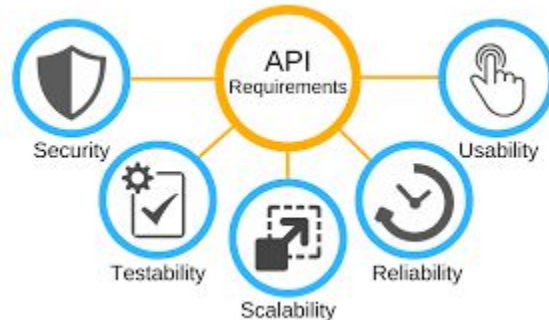
- Extended trips
- Flying
- Hotels
- Rents a car

? Can developer start coding now ?

Are There User Stories For Services?

There's no point in developing a service unless you have some ideas about what kinds of applications and services (clients) might use it and what they would use it for - so there needs to be some way to describe requirements for the API

Services are built for software engineers, to use in their own applications and services, not for users - but we can still adopt user story format and even domain terminology



Example Stories for a Shopping Service

As a visitor shopping app, I want an API that returns the list of nearby shops when I send a location, so that I can refer to these shops



Conditions of Satisfaction:

- The API should accept latitude, longitude and (optional) max-distance
- It should return a list of shops whose entrances are within max-distance (or default) of the requested lat-lon



As a rewards app, I want an API that returns the rewards programs affiliated with a specified shop, so that I can determine whether that shop is relevant

Conditions of Satisfaction:

- The API should accept the name and address of a specific shop
- It should return the list of rewards programs the shop participates in

➤ Notice the service does not itself support shopping, it supports shopping-related apps

Agenda

1. Software Requirements Overview
2. User Stories
3. Use Cases



Use Cases

It's hard to extract enough information from user stories and their conditions of satisfaction to start coding a new project

Use cases expand user stories to describe the step by step details of how a role interacts with the software to exercise the feature and achieve the goal, while trying to capture additional structure, contingencies and alternative flows, and (in the extreme) anything/everything that **CAN GO WRONG**

The term “*use case*” is often used by business to mean “a specific situation in which a product or service could potentially be used”, but here we mean more specifically “description of how an actor who actually uses that process or system will accomplish a goal”

The actor does not need to be human, it is anyone or anything that uses the process or system

Maybe I'll get some
food at that campfire



What could possibly go wrong?

Thousands of years later



What is the Difference between User Story and Use Case?

User stories are about needs. When you write a user story, what you're describing is a “raw” user need. It's something that the user needs to do (or developer needs to use). If you never build any software for them, then that need will still exist! Written in the language of the target users, so any member of the target user group should be able to read a user story and immediately understand what it means

Use cases are about the behavior you'll build into the software to meet those needs. A developer who needs to build working software should be able to read a use case and get a good sense of what the software needs to do. It typically has a lot of detail, and describes everything that the developer needs to build in order to meet the need. That's why it has a lot more detail, and needs to be clear and unambiguous

user story

- ✓ BRIEF AND SIMPLIFIED DESCRIPTION
- ✓ ENCOURAGES CREATIVITY AND DISCUSSION

use case

- ✓ MORE COMPLETE
- ✓ HIGHER LEVEL OF DETAIL
- ✓ MORE CONCRETE

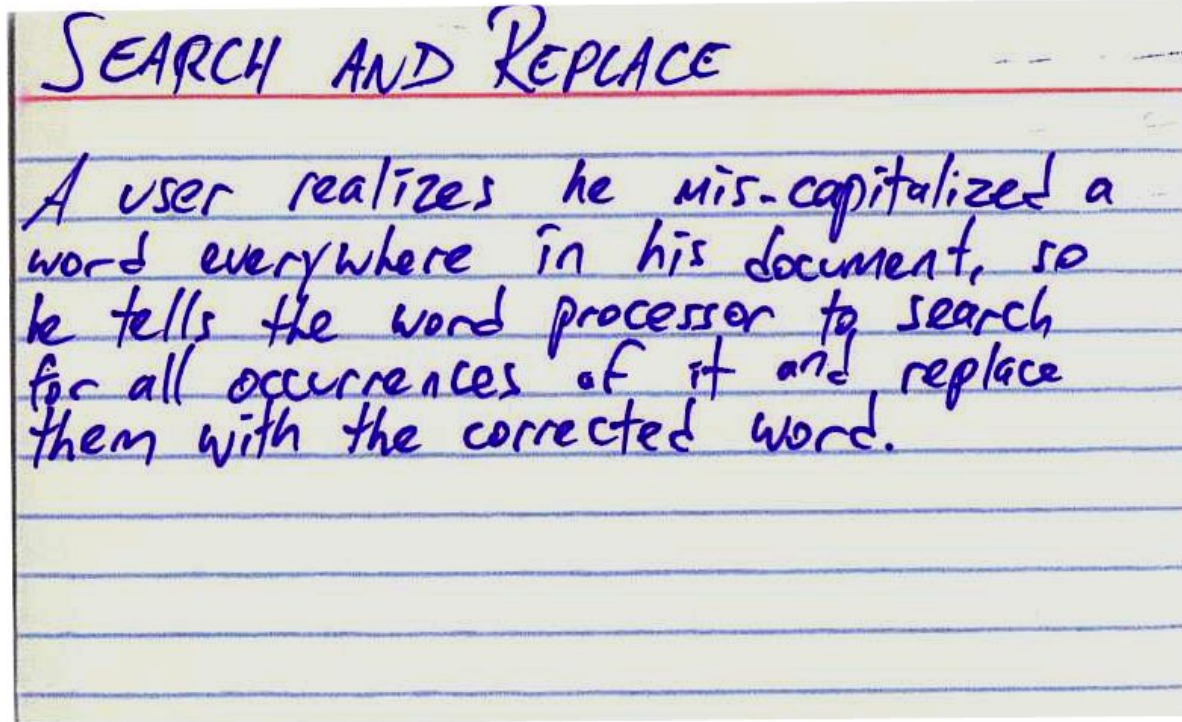
What Could Possibly Go Wrong?

Another difference between user stories and use cases is use cases might not use customer language, particularly for exception flows, since there are things that can go wrong that a customer would not think about

- Invalid input data
- Invalid output data
- Something the feature relies on is down, busy, slow, ...
- Requestor has exceeded limits
- ...



Example User Story



Corresponding Use Case part 1

Name UC-8: Search and Replace

Summary All occurrences of a search term are replaced with replacement text.

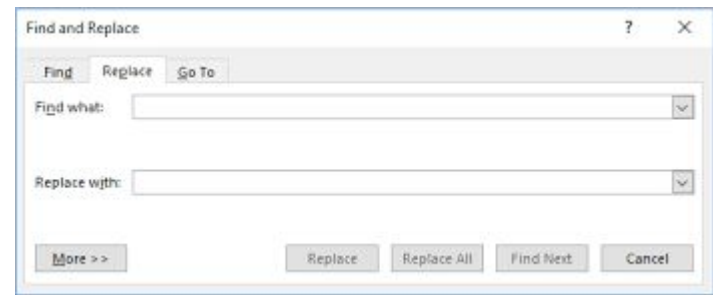
Rationale While editing a document, many users find that there is text somewhere in the file being edited that needs to be replaced, but searching for it manually by looking through the entire document is time-consuming and ineffective. The search-and-replace function allows the user to find it automatically and replace it with specified text. Sometimes this term is repeated in many places and needs to be replaced. At other times, only the first occurrence should be replaced. The user may also wish to simply find the location of that text without replacing it.

Actors All users

Preconditions A document is loaded and being edited.

Basic Course of Events

1. The user indicates that the software is to perform a search-and-replace in the document.
2. The software responds by requesting the search term and the replacement text.
3. The user inputs the search term and replacement text and indicates that all occurrences are to be replaced.
4. The software replaces all occurrences of the search term with the replacement text.

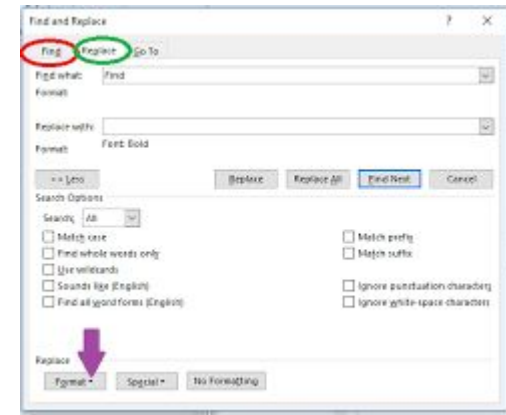


Corresponding Use Case part 2

...

Alternative Paths

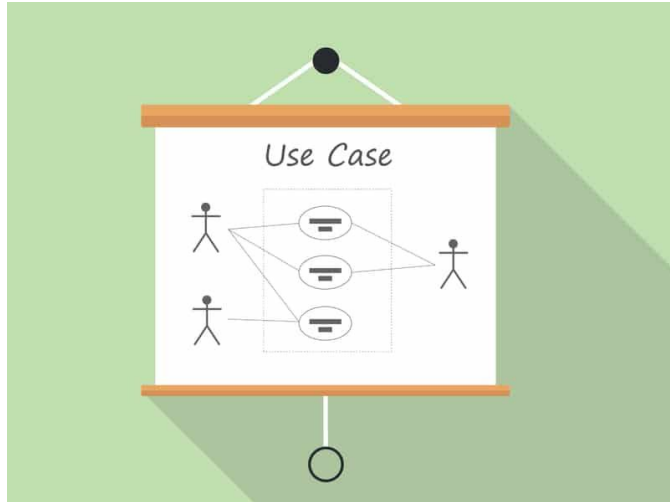
1. In Step 3, the user indicates that only the first occurrence is to be replaced. In this case, the software finds the first occurrence of the search term in the document being edited and replaces it with the replacement text. The postcondition state is identical, except only the first occurrence is replaced, and the replacement text is highlighted.
2. In Step 3, the user indicates that the software is only to search and not replace, and does not specify replacement text. In this case, the software highlights the first occurrence of the search term and the use case ends.
3. The user may decide to abort the search-and-replace operation at any time during Steps 1, 2, or 3. In this case, the software returns to the precondition state.



Postconditions All occurrences of the search term have been replaced with the replacement text.

Not UML Use Case Diagrams

If you google for “use case”, you may find something like this



We mean something more like this

UC-0015	Register Book Loan	
Dependencies	• OBJ-0001 To manage book loans (objective) • OBJ-0005 To know library users' preferences (objective) • CRQ-0003 Maximum number of simultaneous loans (business rule) • CRQ-0014 Return date for a loan (business rule)	
Description	The system shall behave as described in the following use case when a library user requests a loan of one or more books.	
Precondition	The library user has been identified by means of his or her identity card, has picked up the books to loan from the shelves, has not reached the maximum number of simultaneous loans and has no penalty.	
Ordinary Sequence	Step	Action
	1	Actor librarian requests the system for starting the book loan registering process.
	2	The system requests for the identification of the library user requesting a loan.
	3	Actor librarian provides identification data of the library user to the system.
	4	The system requests for the identification of the books to be loaned.
	5	Actor librarian provides identification data of the books to be loan to the system.
	6	The system displays the return date for each of the books to be loan and requests loan confirmation for each of them.
	7	Actor library user confirms the librarian which books he or she wants to loan after knowing return dates.
	8	Actor librarian re-confirms the book loans confirmed by the library user to the system.
	9	The system informs that the book loans have been successfully registered.
Postcondition	The library user can take the loaned books away and the system has registered the book loans.	
Exceptions	Step	Action
	3	If the library user has already reached the maximum number of simultaneous loans or has a penalty, the system informs of the situation, then this use case is cancelled.
Comments	The maximum number of simultaneous book loans and the loan period depend on the library policy and can change in the future. See business rules CRQ-0003 y CRQ- 0014.	

Use Cases are More Structured than User Stories

Depending on how in-depth and complex you want or need to get, use cases describe a combination of the following elements:

- **Actor** – anyone or anything that performs a behavior (who is using the system)
- **Stakeholder** – someone or something with vested interests in the behavior of the system
- **Primary Actor** – stakeholder who initiates an interaction with the system to achieve a goal
- **Preconditions/Postconditions** – what must be true or happen before and after the use case runs
- **Triggers** – this is the event that causes the use case to be initiated
- **Main success scenario(s)** – aka basic flow or happy path (not necessarily literally a single execution path)
- **Alternative flows** – special cases
- **Exception flows** – alternative flows for error cases

[Use case templates](#)

Example Use Case

1. On Wednesdays, the housekeeper reports to the laundry room.
2. The housekeeper sorts the laundry that is there.
3. Then they wash each load.
4. They dry each load.
5. A. They fold the items that need folding. [← basic flow](#)
B. They iron and hang the items that are wrinkled. [← alternative flow](#)
C. They throw away any laundry item that is irrevocably shrunken, soiled or scorched. [← alternative flow](#)

Actors: Housekeeper, system that transports laundry to the laundry room.

Stakeholders: Owners/wearers of the laundry items.

Trigger: Dirty laundry has been transported to the laundry room.

Precondition: It is Wednesday.

Postcondition: Clean laundry is folded or hung. Damaged laundry is discarded.



Example - Further Details and Possible Exception Flows



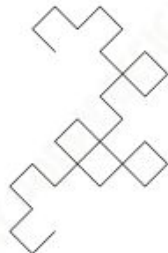
1. A. The housekeeper reports to the laundry room.
B. The housekeeper does not show up. $\Leftarrow$ **exception flow**
C. The laundry room is flooded (or other reasons the housekeeper cannot get to the laundry room). $\Leftarrow$ **exception flow**
2. A. The housekeeper sorts the laundry that is there into hot, warm, cold wash (or other categories). $\Leftarrow$ **more detail needed**
B. The laundry is missing. $\Leftarrow$ **exception flow**
3. A. Then they wash each load (in what order? what detergent, etc. is placed in washer?). $\Leftarrow$ **more detail needed**
B. The washing machine does not work. $\Leftarrow$ **exception flow**
C. The xxx load is too large for the washing machine and needs to be split (etc). $\Leftarrow$ **more detail needed / exception**
D. Washing machine takes too long to do all the loads before the housekeeper has to leave. $\Leftarrow$ **exception flow**
4. And so on....

Upcoming Assignments

- [First Iteration](#) past due
- [First Iteration Demo](#) due this Thursday, October 26, by 11:59pm

First Assessment will be 12:01am Friday October 27 through 11:59pm Monday October 30

- [Second Iteration](#) due Thursday, November 30, by 11:59pm
- [Second Iteration Demo](#) due Thursday, December 7, by 11:59pm



Thursday

Last Chance questions before First Assessment

From Requirements to Code: CRC (Class Responsibilities Collaborators) Design

Class Name	
Responsibilities	Collaborators

Ask Me Anything